DingIt Platform - Technical Assessment Report

Prepared by: Rupesh Jain **Organization:** Uniworld Technologies **Date:** January 20, 2026
Version: 1.0 **Classification:** Confidential

Table of Contents

---

1. Executive Summary

Overview

DingIt is a service marketplace platform enabling customers to discover service providers, book services, submit reviews, and tip service members. The application supports Web, iOS, and Android platforms through a hybrid mobile architecture.

Key Findings

Category	Status	Risk Level
Security	Requires Attention	HIGH
Test Coverage	Critical Gap	HIGH
Code Quality	Good Foundation	MEDIUM
Documentation	Incomplete	MEDIUM
Architecture	Well-Structured	LOW
CI/CD Pipeline	Operational	LOW

Critical Issues Requiring Immediate Attention

1. **JWT Verification Disabled** on payment webhook endpoint
2. **Overly Permissive CORS Policy** allowing any origin
3. **Zero Test Coverage** (only 1 test file exists)
4. **No Logging/Monitoring Infrastructure**

Overall Assessment

The DingIt platform demonstrates solid architectural decisions with modern technology choices. However, several security vulnerabilities and quality gaps must be addressed before scaling to production workloads. The codebase requires immediate attention to security configurations and comprehensive test coverage implementation.

---

2. Project Overview

Application Purpose

DingIt is a marketplace connecting customers with service providers. Core features include:

- Service provider discovery with geolocation
- Product/service catalog management
- Customer review and rating system
- Tipping system for service members

- Stripe-integrated payment processing
- Subscription management for service providers

Platform Support

Platform	Technology	Status
Web	Angular SPA	Production
iOS	Capacitor Native	Production
Android	Capacitor Native	Production

Environments

Environment	URL	Purpose
Production	https://app.2dingit.com	Live users
Development	https://dev.app.2dingit.com	Staging/Testing
Local	http://localhost:4200	Developer workstations

---

3. Implemented Features

This section provides an overview of all features currently implemented in the DingIt platform with application screenshots.

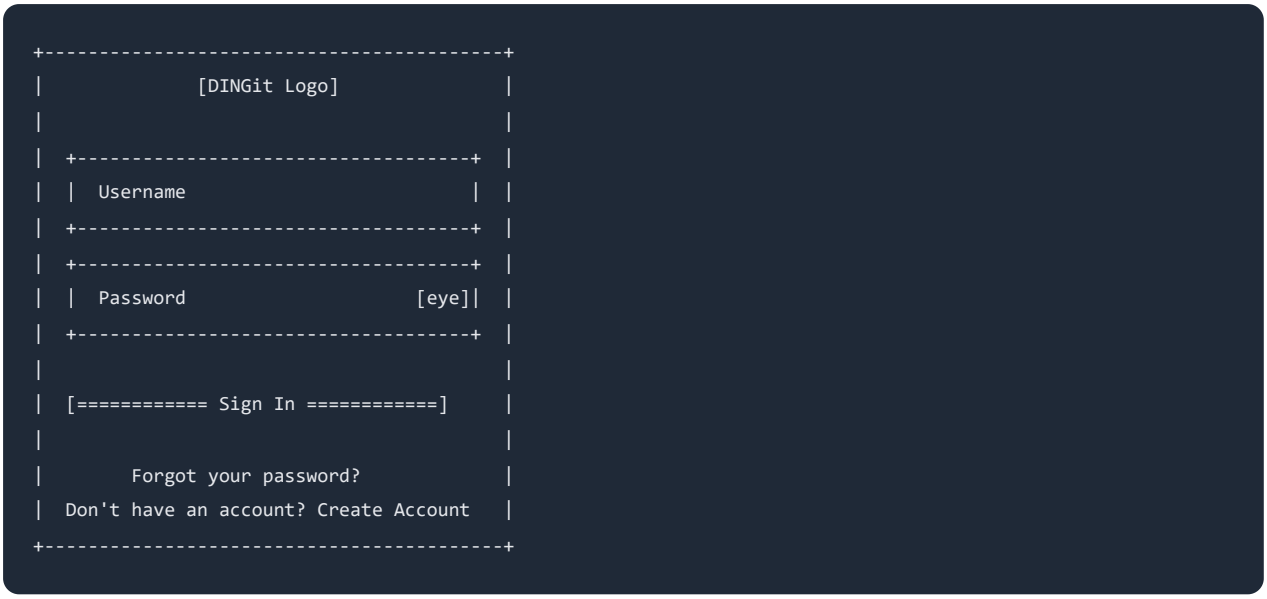
3.1 Authentication & User Management

Login Screen

The login interface provides secure authentication with:

- Email/username and password fields
- Password visibility toggle
- "Forgot your password?" recovery link
- "Create Account" registration link

Screenshot: Login Page



Sign Up Screen

New user registration with:

- Email address field
- Password field
- Confirm password field
- Sign Up button

3.2 Home Page & Discovery

The home page serves as the main discovery interface featuring:

Monthly Spotlight

- Featured business banner with image
- Business name and description
- Location information

Search Functionality

- Prominent search bar
- Real-time search results
- Location-aware distance display

Businesses Carousel

- Horizontally scrollable business cards
- Each card displays:
  - Business thumbnail image
  - Business name
  - Description/subtitle
  - Distance from user (e.g., "336mi. away")
  - Star rating (1-5 scale)

Top-rated Providers

- Vertical list of highest-rated businesses
- Large promotional images
- Business details and ratings

Bottom Navigation

- Home icon (current page)
- Money icon (financial features)
- Settings icon (app settings)
- Profile icon (user profile)

3.3 Service Provider Detail Page

Comprehensive business profile view including:

Header Section

- Large header/promotional image
- Business name and subtitle
- Star rating display (5-star scale)
- Quick action buttons:
  - **Call**: Direct phone call to business
  - **Website**: Open business website
  - **Share**: Copy link to clipboard

Featured Product

- "Most Popular" product showcase
- Product image and name
- "See Full Menu" button

Active Promotions

- Promotional banner display
- Direct link to promotional content

Location & Navigation

- Interactive Google Maps display
- Business location marker
- **Navigate** button for directions
- Distance display (e.g., "335.6 mi away")
- Full address with street, city, state, zip

Top Rated Employees

- Team member photos (square avatars)
- Individual names (clickable profiles)
- Star ratings per member
- Top 3 displayed on main page

Reviews Section

- Customer reviews with:
  - Reviewer name and photo
  - Star rating
  - Review text
- "Leave a review" button
- "See All" link for complete list

Operating Hours

- Day-by-day schedule
- Open and close times
- Support for multiple time slots

3.4 Product Menu

Product catalog page displaying:

- Product photo thumbnails
- Product name
- Product description
- Price(s) column
- Table layout for easy scanning

3.5 Search Results

Search functionality with:

- Search input field
- Results showing:
  - Business thumbnail
  - Business name
  - Description
  - Distance (e.g., "345.4mi. away")
  - Location (City, State)

3.6 Review & Rating System

Review Features

- Overall provider rating (1-5 stars)
- Individual team member ratings
- Individual product ratings
- Written description/comments
- Low-rating comment requirement

Tipping System

- Per-team-member tip amounts
- Stripe checkout integration
- Tips for onboarded team members only

3.7 Financial Management

For Service Provider Owners

- Subscription management via Stripe
- Free trial support
- Billing portal access
- Multi-provider subscription quantity

For Service Team Members

- Stripe Express account onboarding
- Tip collection and transfers
- Transfer history dashboard
- Earnings tracking

3.8 Settings & Administration

User Settings

- Profile management
- Password changes
- Account deletion

Service Provider Management

- Create/edit service providers
- Image management (header, promo, products)
- Operating hours configuration
- Team member invitations
- Product catalog management

3.9 Feature Summary Matrix

Category	Feature	Status
Authentication	Email/Password Login	Implemented
	User Registration	Implemented
	Password Reset	Implemented

	Profile Management	Implemented
Discovery	Home Page	Implemented
	Monthly Spotlight	Implemented
	Search	Implemented
	Location Services	Implemented
Service Providers	Provider Profiles	Implemented
	Product Menus	Implemented
	Operating Hours	Implemented
	Team Management	Implemented
	Google Maps Integration	Implemented
Reviews	View Reviews	Implemented
	Create Reviews	Implemented
	Rating System	Implemented
	Tipping	Implemented
Payments	Stripe Integration	Implemented
	Subscriptions	Implemented
	Team Member Payouts	Implemented
Mobile	iOS App	Production
	Android App	Production
	Web App	Production

---

4. Technology Stack

Frontend Application

Component	Technology	Version	Notes
Framework	Angular	17.0.0	Standalone Components API
UI Framework	Tailwind CSS	3.4.7	Utility-first CSS
Component Library	DaisyUI	4.7.3	Tailwind component library
Mobile Framework	Capacitor	6.1.2	Cross-platform native
Language	TypeScript	5.2.2	Strict type checking
State Management	RxJS	7.8.0	Reactive patterns
Maps	Google Maps API	Latest	@angular/google-maps
Build Tool	Angular CLI	17.3.1	Vite-based build

Backend Infrastructure

Component	Technology	Version	Notes
Backend-as-a-Service	Supabase	Latest	PostgreSQL + Auth + Storage
Database	PostgreSQL	15	With PostGIS extension
Edge Functions	Deno	Latest	TypeScript serverless
Authentication	Supabase Auth	Built-in	JWT-based
File Storage	Supabase Storage	Built-in	S3-compatible

Third-Party Integrations

Service	Purpose	API Version
Stripe	Payments & Subscriptions	2024-06-20
Google Maps	Location & Mapping	Latest

Development Tools

Tool	Purpose	Version
Karma	Test Runner	6.4.0
Jasmine	Test Framework	5.1.0
GitHub Actions	CI/CD	Latest
Supabase CLI	Database Management	1.148.6

---

4. Architecture Analysis

Application Structure

```
DingIt/
├── app/                          # Angular Frontend (6.5MB)
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/       # 20+ reusable components
│   │   │   ├── guards/           # 3 route guards
│   │   │   ├── pages/            # 8 major page sections
│   │   │   └── services/         # 8 injectable services
│   │   ├── environments/         # 3 environment configs
│   │   └── types/                # TypeScript definitions
│   ├── android/                  # Capacitor Android
│   └── ios/                      # Capacitor iOS
├── supabase/                     # Backend Infrastructure
│   ├── functions/                # 5 Edge Functions
│   │   ├── stripe/               # Payment operations
│   │   ├── stripe-webhook/       # Payment events
│   │   ├── delete-user/          # User management
│   │   ├── invite-service-member/
│   │   └── upload-thumbnail/
│   └── migrations/               # 20+ database migrations
├── graphics/                     # Design assets
└── .github/workflows/            # CI/CD pipelines
```

Data Flow Architecture

```
┌─────────────────┐    ┌─────────────────┐    ┌─────────────────┐
│   Angular App   │───▶│    Supabase     │───▶│   PostgreSQL    │
│   (Frontend)    │    │   (API Layer)   │    │   (Database)    │
└─────────────────┘    └─────────────────┘    └─────────────────┘
        │                      │
```

Strengths

- **Modern Angular 17** with standalone components reduces boilerplate
- **Capacitor 6** provides excellent native mobile integration
- **Supabase** offers real-time capabilities and simplified backend
- **Clean separation** of concerns between frontend and backend
- **Type-safe** end-to-end with generated TypeScript definitions

Areas of Concern

- No API rate limiting implementation
- No caching layer for frequently accessed data
- Database connection pooling disabled

---

5. Security Assessment

5.1 CRITICAL: JWT Verification Disabled on Webhook

Severity: CRITICAL **Location:** `supabase/config.toml` (Line 161-162) **CVSS Score Estimate:** 7.5 (High)

Current Configuration

```
[functions.stripe-webhook]
verify_jwt = false
```

Risk Analysis

With JWT verification disabled, the webhook endpoint accepts requests from any source without authentication. While Stripe signature validation is implemented in the code, this configuration:

- Allows reconnaissance attacks to probe the endpoint
- Increases attack surface unnecessarily
- Violates defense-in-depth security principles

Mitigating Factor

The webhook does implement Stripe signature verification:

```
// stripe-webhook/index.ts (Lines 27-37)
try {
  event = await stripe.webhooks.constructEventAsync(
    body,
    signature!,
    webhookKey!.value,
    undefined
  );
} catch (err) {
  console.log(`❌ Error message: ${err.message}`);
  return new Response(err.message, {status: 400});
}
```

Recommendation

Document that Stripe signature validation is the primary security control, or implement IP allowlisting for Stripe webhook IPs.

5.2 HIGH: Overly Permissive CORS Policy

Severity: HIGH **Location:** `supabase/functions/_shared/cors.ts`

Current Configuration

```
export const corsHeaders = {
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}
```

Risk Analysis

The wildcard `*` origin allows **any website** to make cross-origin requests to the API endpoints. This creates vulnerabilities for:

Attack Vector	Description	Impact
CSRF	Malicious sites can trigger authenticated requests	User data modification
Data Exfiltration	Attackers can read API responses	Information disclosure
Session Hijacking	Combined with XSS enables token theft	Account takeover

Example Attack Scenario

```
// Attacker's website (evil-site.com)
fetch('https://your-supabase-project.functions.supabase.co/stripe', {
  method: 'POST',
  headers: {
    'Authorization': 'Bearer STOLEN_TOKEN',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ action: 'getTransfers' })
})
.then(response => response.json())
.then(data => {
  // Attacker now has user's financial transfer data
  sendToAttackerServer(data);
});
```

Recommended Configuration

```
const ALLOWED_ORIGINS = [
  'https://app.2dingit.com',
  'https://dev.app.2dingit.com',
  'http://localhost:4200'  // Only for development
];

export const corsHeaders = (origin: string) => ({
  'Access-Control-Allow-Origin': ALLOWED_ORIGINS.includes(origin) ? origin : '',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
  'Access-Control-Allow-Credentials': 'true'
});
```

5.3 MEDIUM: No Input Validation on Edge Functions

Severity: MEDIUM **Location:** `supabase/functions/stripe/index.ts`

Current Code (No Validation)

```
const input = await req.json();
if(input.action == 'onboard') {
  // Proceeds directly without validating input structure
  let serviceMemberUser = userObj?.service_member_user;
  // ...
}
```

Risk Analysis

Without input validation:

- Malformed requests may cause unexpected errors
- Type coercion attacks possible
- Denial of service through resource exhaustion

Recommended Pattern

```
import { z } from 'npm:zod';

const OnboardSchema = z.object({
  action: z.literal('onboard'),
});

const TipSchema = z.object({
  action: z.literal('tip'),
  review: z.string().uuid(),
});

const InputSchema = z.discriminatedUnion('action', [
  OnboardSchema,
  TipSchema,
  // ... other actions
]);

// Usage
const parseResult = InputSchema.safeParse(await req.json());
if (!parseResult.success) {
  return new Response(JSON.stringify({ error: 'Invalid input' }), { status: 400 });
```

```
}
const input = parseResult.data;
```

5.4 MEDIUM: No Rate Limiting

Severity: MEDIUM **Location:** All Edge Functions

Risk Analysis

Without rate limiting, the API is vulnerable to:

- Brute force attacks
- Denial of service attacks
- Resource exhaustion
- Increased Stripe API costs from repeated calls

Recommendation

Implement rate limiting using Supabase's built-in capabilities or a service like Upstash Redis:

```
import { Ratelimit } from 'npm:@upstash/ratelimit';
import { Redis } from 'npm:@upstash/redis';

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(10, '60 s'), // 10 requests per minute
});

// In request handler
const identifier = authUser.id;
const { success, limit, reset, remaining } = await ratelimit.limit(identifier);

if (!success) {
  return new Response('Rate limit exceeded', { status: 429 });
}
```

---

6. Code Quality Analysis

6.1 CRITICAL: Test Coverage Deficiency

Current State: ~0% coverage (1 test file exists)

Existing Test File Analysis

```
// app.component.spec.ts - OUTDATED
it('should render title', () => {
  const fixture = TestBed.createComponent(AppComponent);
  fixture.detectChanges();
  const compiled = fixture.nativeElement as HTMLElement;
  expect(compiled.querySelector('h1')?.textContent).toContain('Hello, app');
});
```

Issue: The test expects `Hello, app` in an `h1` tag, but the actual component renders only `<router-outlet />`. This test will **fail** if executed.

Actual Component

```
// app.component.ts
@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html',  // Contains: <router-outlet />
})
export class AppComponent {
  title = 'DINGit';  // Note: 'DINGit' not 'app'
  // ...
}
```

Missing Test Coverage

Area	Files	Test Files	Coverage
Services	8	0	0%
Components	20+	0	0%
Guards	3	0	0%
Pages	15+	0	0%
Edge Functions	5	0	0%

Required Test Categories

Unit Tests (Target: 100%)

```
// Example: stripe.service.spec.ts
describe('StripeService', () => {
  it('should handle onboarding flow', async () => {
    // Test Stripe Connect account creation
  });

  it('should process tip checkout', async () => {
    // Test checkout session creation
  });

  it('should retrieve transfers', async () => {
    // Test transfer history retrieval
  });
});
```

Integration Tests

```
// Example: review-flow.spec.ts
describe('Review Creation Flow', () => {
  it('should create review with service member ratings', async () => {
    // Test complete review submission
  });

  it('should calculate overall rating correctly', async () => {
    // Test rating algorithm
  });
});
```

E2E Tests

```
// Example: e2e/checkout.spec.ts
describe('Payment Flow', () => {
  it('should complete subscription checkout', async () => {
    // Navigate to subscription page
    // Click subscribe button
    // Complete Stripe checkout
    // Verify subscription active
  });
});
```

6.2 MEDIUM: No Code Comments

Location: Throughout codebase

Example: Complex Business Logic Without Comments

```
// stripe-webhook/index.ts (Lines 69-77)
// No explanation of tip calculation logic or fee structure
tips?.forEach(async (t) => {
  await stripe.transfers.create({
    amount: Math.floor(t.amount * 97.1) - 30,  // What does 97.1 and 30 represent?
    currency: 'usd',
    destination: t.account_id,
    transfer_group: reviewId,
    source_transaction: event.data.object.id
  });
});
```

Questions This Code Raises

1. Why `97.1`? (Appears to be 2.9% platform fee)
2. Why subtract `30`? (Appears to be $0.30 fixed fee)
3. What is the total fee structure?
4. How was this calculation determined?

Recommended Documentation

```
/**
 * Creates Stripe transfers to service members from tip payments.
 *
 * Fee Structure:
 * - Platform fee: 2.9% of tip amount
 * - Fixed fee: $0.30 per transaction
 * - Formula: floor(tip * 0.971) - 30 cents
 *
 * Example: $10.00 tip
 * - Gross: 1000 cents
 * - After percentage: floor(1000 * 0.971) = 971 cents
 * - After fixed fee: 971 - 30 = 941 cents = $9.41
 * - Service member receives: $9.41
 *
 * @see https://stripe.com/docs/connect/charges-transfers
 */
tips?.forEach(async (t) => {
  const PLATFORM_FEE_MULTIPLIER = 0.971;  // 2.9% platform fee
  const FIXED_FEE_CENTS = 30;              // $0.30 per transaction

  const grossAmountCents = t.amount * 100;
  const afterPercentageFee = Math.floor(grossAmountCents * PLATFORM_FEE_MULTIPLIER);
  const netAmountCents = afterPercentageFee - FIXED_FEE_CENTS;
```

```
  await stripe.transfers.create({
    amount: netAmountCents,
    currency: 'usd',
    destination: t.account_id,
    transfer_group: reviewId,
    source_transaction: event.data.object.id
  });
});
```

6.3 MEDIUM: Incomplete Documentation

Current README.md:

```
# DingIt
Sources for the DingIt web application

## Schema
[Entity Relationship Diagram]

## Development
supabase start
supabase db diff -f <diff_file_title>
```

Missing Documentation:

- Project setup instructions
- Environment variable reference
- API endpoint documentation
- Deployment procedures
- Contributing guidelines
- Architecture decision records

6.4 LOW: No Logging Infrastructure

Current State: Only `console.log()` statements

```
// Current logging approach
console.log(acct)
console.log(serviceMemberUser)
console.log('account.updated for ' + event.data.object.id)
```

Required: Structured logging with Serilog/DatadogHQ integration per project standards.

Recommended Implementation

```
import { Logger } from './logger';

const logger = new Logger({
  service: 'stripe-webhook',
  environment: Deno.env.get('ENVIRONMENT'),
  datadogApiKey: Deno.env.get('DD_API_KEY')
});

// Structured logging
logger.info('Processing webhook event', {
  eventType: event.type,
  eventId: event.id,
  accountId: event.data.object.id
});

logger.error('Failed to process transfer', {
  error: err.message,
  reviewId,
  tipAmount: t.amount
});
```

---

7. Performance Considerations

7.1 Database Connection Pooling Disabled

Location: `supabase/config.toml` (Lines 27-37)

```
[db.pooler]
enabled = false
```

Impact

Without connection pooling:

- Each request creates a new database connection
- Connection overhead adds latency
- May hit PostgreSQL connection limits under load
- Resource inefficiency

Recommendation

Enable for production:

```
[db.pooler]
enabled = true
pool_mode = "transaction"
default_pool_size = 20
max_client_conn = 100
```

7.2 No Caching Strategy

Current State: All data fetched directly from database

Cacheable Data Examples

Data Type	Cache Duration	Current Behavior
Service Provider List	5 minutes	Direct DB query
Product Catalog	5 minutes	Direct DB query
User Profile	1 minute	Direct DB query
Settings	10 minutes	Direct DB query

Recommendation

Implement caching at the Edge Function level:

```
// Using Supabase Edge Function caching
const cacheKey = `service-providers:${region}`;
const cached = await kv.get(cacheKey);

if (cached) {
  return new Response(cached, { headers: { 'X-Cache': 'HIT' } });
}

const data = await supabase.from('service_provider').select('*');
await kv.set(cacheKey, JSON.stringify(data), { ex: 300 }); // 5 min TTL
```

```
return new Response(JSON.stringify(data), { headers: { 'X-Cache': 'MISS' } });
```

---

8. Detailed Findings

Issue Summary Table

ID	Severity	Category	Issue	Location
SEC-001	CRITICAL	Security	JWT verification disabled on webhook	`supabase/config.toml:161`
SEC-002	HIGH	Security	Permissive CORS policy	`_shared/cors.ts:2`
SEC-003	MEDIUM	Security	No input validation	`stripe/index.ts`
SEC-004	MEDIUM	Security	No rate limiting	All Edge Functions
QA-001	CRITICAL	Quality	Zero test coverage	Project-wide
QA-002	MEDIUM	Quality	No code comments	Project-wide
QA-003	MEDIUM	Quality	Incomplete documentation	`README.md`
QA-004	MEDIUM	Quality	No logging infrastructure	Project-wide
QA-005	LOW	Quality	Outdated test file	`app.component.spec.ts`
PERF-001	MEDIUM	Performance	DB pooling disabled	`config.toml:28`
PERF-002	LOW	Performance	No caching strategy	Project-wide
PERF-003	LOW	Performance	Hardcoded trial period	`stripe/index.ts:204`

---

9. Recommendations

Immediate Actions (Week 1-2)

1. Security Hardening

- [ ] Document Stripe signature validation as primary security control OR re-enable JWT
- [ ] Implement domain-restricted CORS policy
- [ ] Add input validation using Zod schema validation
- [ ] Implement rate limiting on Edge Functions

2. Test Infrastructure

- [ ] Fix existing `app.component.spec.ts` test
- [ ] Set up test coverage reporting
- [ ] Create test templates for services, components, and Edge Functions

Short-Term Actions (Week 3-4)

3. Logging & Monitoring

- [ ] Implement structured logging with DatadogHQ
- [ ] Add error tracking and alerting
- [ ] Create dashboard for key metrics

4. Documentation

- [ ] Create comprehensive setup guide
- [ ] Document all environment variables
- [ ] Add inline code comments to complex logic
- [ ] Create API documentation

Medium-Term Actions (Month 2)

5. Test Coverage Implementation

- [ ] Unit tests for all 8 services (Target: 100%)
- [ ] Unit tests for all guards (Target: 100%)

- [ ] Component tests for critical components
- [ ] Integration tests for payment flows
- [ ] E2E tests for user journeys

6. Performance Optimization

- [ ] Enable database connection pooling
- [ ] Implement caching strategy
- [ ] Add performance monitoring

Priority Matrix

```
                IMPACT
                High    |    Low
            ┌───────────┼──────────┐
      High  |  SEC-001  |  QA-003  |
            |  SEC-002  |  PERF-003|
URGENCY     |  QA-001   |          |
            ├───────────┼──────────┤
      Low   |  SEC-003  |  PERF-001|
            |  SEC-004  |  PERF-002|
            |  QA-002   |  QA-005  |
            |  QA-004   |          |
            └───────────┴──────────┘
```

---

10. Appendix

A. Dependency Versions

Frontend Dependencies

```
{
  "@angular/core": "^17.0.0",
  "@angular/animations": "^17.3.12",
  "@angular/cdk": "^17.3.8",
  "@angular/google-maps": "^17.3.4",
  "@capacitor/core": "^6.1.2",
  "@capacitor/android": "^6.1.2",
  "@capacitor/ios": "^6.1.2",
  "@supabase/supabase-js": "^2.39.8",
  "rxjs": "~7.8.0",
  "tailwindcss": "^3.4.7",
  "daisyui": "^4.7.3",
```

```
  "typescript": "~5.2.2"
}
```

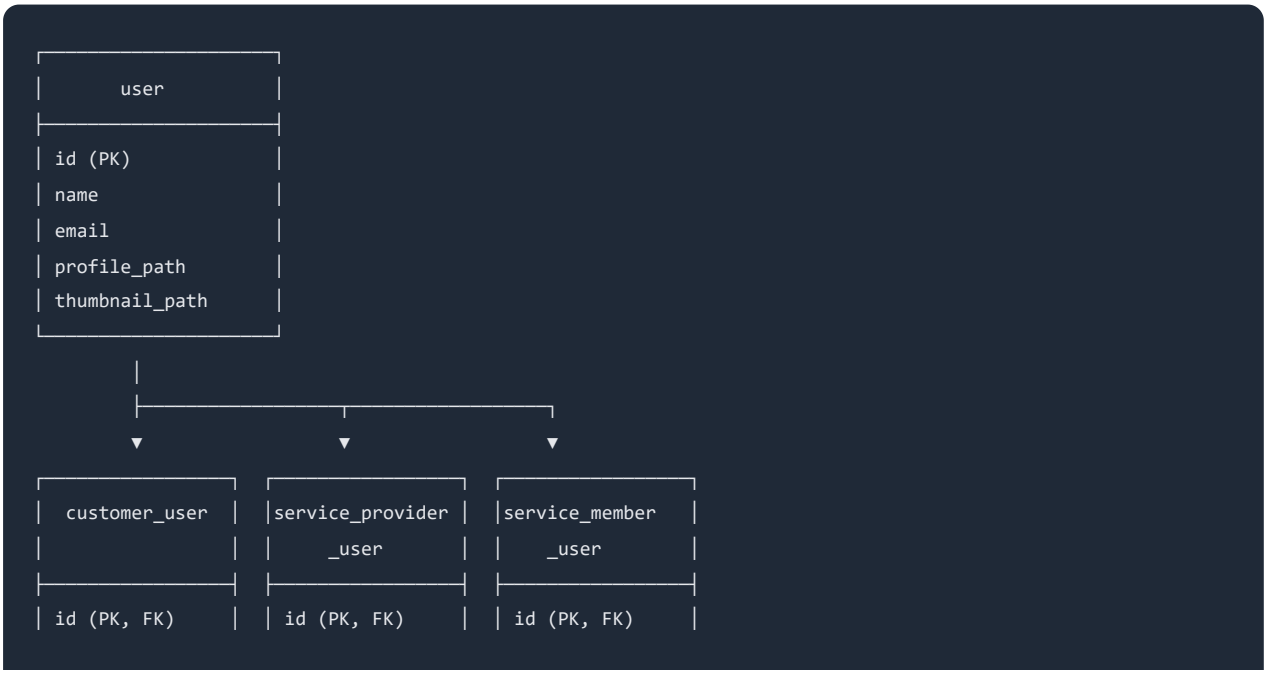
Backend Dependencies

```
// Deno imports
import Stripe from "npm:stripe@^16.0.0"
import { createClient } from 'jsr:@supabase/supabase-js@2'
```

B. Environment Variables Reference

Variable	Description	Required
`SUPABASE_URL`	Supabase project URL	Yes
`SUPABASE_ANON_KEY`	Supabase anonymous key	Yes
`SUPABASE_SERVICE_ROLE_KEY`	Supabase admin key	Yes (Edge Functions)
`STRIPE_KEY`	Stripe secret key	Yes (via settings table)
`STRIPE_WEBHOOK_SECRET`	Webhook signing secret	Yes (via settings table)
`APP_URL`	Application base URL	Yes (via settings table)

C. Database Schema (Key Tables)



```
|                |  | stripe_customer |  | stripe_account  |
|                |  | stripe_sub_id   |  | onboarded       |
|                |  | active          |  |                 |
└────────────────┘  └─────────────────┘  └─────────────────┘
```

D. CI/CD Pipeline Overview

```
# Deployment Flow
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│  Push to     │────▶│   GitHub     │────▶│   Deploy     │
│  Branch      │     │   Actions    │     │   Server     │
└──────────────┘     └──────────────┘     └──────────────┘
                           │
                           ▼
                     ┌──────────────┐
                     │   Supabase   │
                     │   Deploy     │
                     └──────────────┘

# Steps:
1. Checkout code
2. Setup Supabase CLI
3. Push database migrations
4. Install npm dependencies
5. Generate TypeScript types
6. Build Angular application
7. Deploy Edge Functions
8. SSH deploy to server
9. Create symbolic link
```

Document Control

Version	Date	Author	Changes
1.0	2026-01-20	Rupesh Jain, Uniworld Technologies	Initial assessment

---